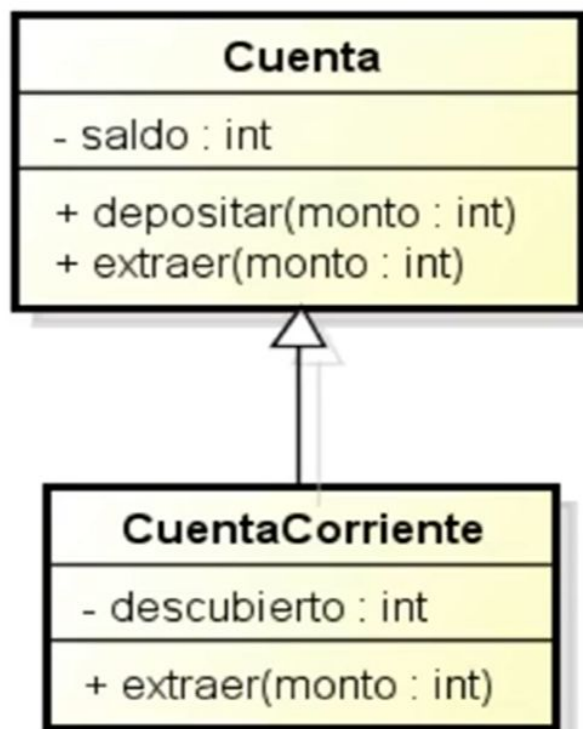


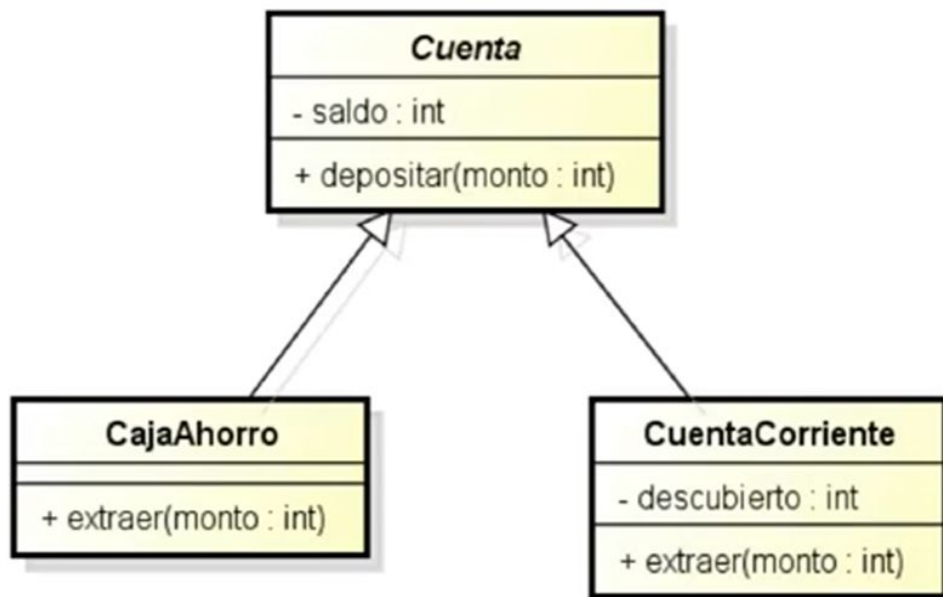
Redefinición

Para implementar de modo diferente la respuesta a un mismo mensaje



Clases abstractas

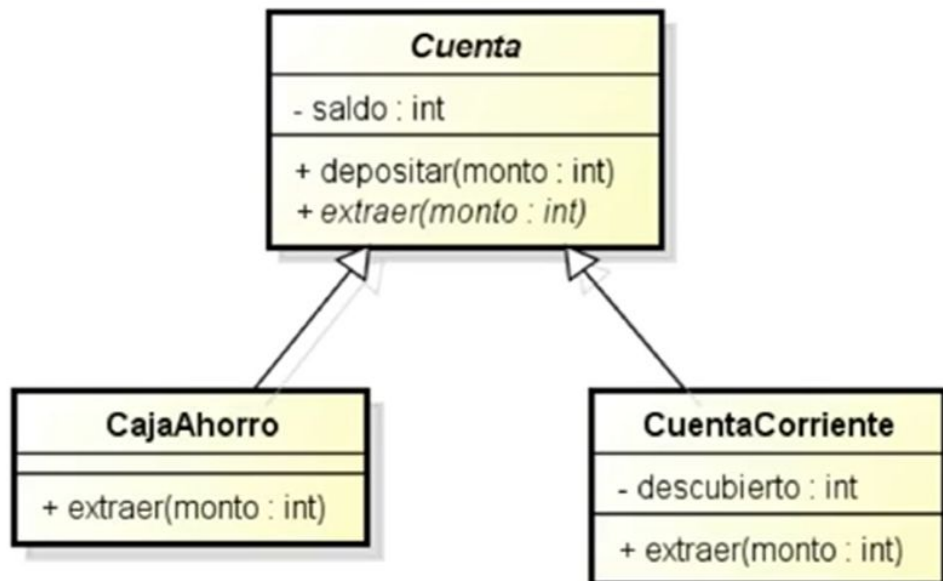
No pueden tener instancias en forma directa
Habitualmente porque sus descendientes cubren todos los casos posibles



Métodos abstractos

No lo implementamos en una clase

Pero deseamos que todas las clases descendientes puedan entender el mensaje



Herencia en Smalltalk y Java

Smalltalk:

```
ColeccionCeldas subclass: #Fila
```

Java:

```
class Fila extends ColeccionCeldas  
{...}
```

En ambos casos hay una clase madre por defecto, llamada Object

En Smalltalk hay que declararla:

```
Object subclass: #ColeccionCeldas
```

Visibilidad

Importante para garantizar encapsulamiento

Atributos y métodos privados

Sólo los puede usar el objeto receptor en su clase

Atributos y métodos públicos

Se los puede usar desde cualquier lado

Atributos y métodos protegidos

Sólo los puede usar el objeto receptor en su clase y en las clases derivadas



Visibilidad en Smalltalk y Java

Smalltalk

Todos los métodos son públicos

Todos los atributos son protegidos

Aunque se recomienda considerarlos privados

Hay convenciones para hacer métodos y atributos privados

¡Ojo que es una solución frágil!: depende de la memoria y la buena voluntad

Java

Visibilidad de atributos, métodos, clases

Directivas *public*, *private* y *protected*

Además hay visibilidad *de paquete*



Clases y métodos abstractos en Java

Directiva *abstract*

```
public abstract void extraer (int  
    monto);
```

```
public abstract class Cuenta {  
    ...  
}
```

Toda clase con un método abstracto debe ser abstracta

Todo se chequea en tiempo de compilación

Clases y métodos abstractos en Smalltalk

Método abstracto

```
extraer: monto  
    self subclassResponsibility.
```

En caso de invocar este método, se lanza una excepción *SubclassResponsibility*

Clase abstracta = clase con un método abstracto

Todo se chequea en tiempo de ejecución

Inicialización y herencia

```
public CuentaCorriente (String titular, int descubierto) {  
    this.titular = titular;  
    this.saldo := 0;  
    this.descubierto := descubierto;  
}
```

```
public CuentaCorriente (String titular, int descubierto)  
{  
    // "super" invoca al constructor de la clase madre  
    super(titular);  
    this.descubierto := descubierto;  
}
```

Inicialización y asociación

¿Es lo mismo? ¿Debemos inicializar en cascada?

Si por cada objeto que creamos, inicializamos en cascada los objetos referenciados: quedarían atados al objeto de arranque de la asociación

Eso se llama composición

No es lo que deseamos en todos los casos

Herencia y comprobación estática

```
Cuenta c = new Cuenta ( );
```

```
CuentaCorriente cc = new  
    CuentaCorriente ( );
```

```
Cuenta c = cc;
```

```
Cuenta ccc = new CuentaCorriente ( );
```

¿Qué comprobaciones hace el compilador?

Recapitulación: preguntas

- ¿Para qué sirve un método abstracto?
- ¿Qué hay que hacer al inicializar en contextos de herencia?
- ¿Es lo mismo para las asociaciones?



Claves

Hay varias maneras de delegar

Y al menos dos maneras de programar por diferencia

Herencia si se va a reutilizar la interfaz tal como está

Relaciones “es un”

Delegación cuando se va a reutilizar cambiando la interfaz

Redefinición permite cambiar implementación manteniendo la semántica

Clases abstractas no tienen instancias

Lecturas obligatorias

"The Art of Enbugging", Andy Hunt y Dave Thomas:

https://media.pragprog.com/articles/jan_03_enbug.pdf

"GetterEradiator", Martin Fowler:

<https://martinfowler.com/bliki/GetterEradiator.html>



Qué sigue

Polimorfismo

Refactorización

Profundización

UML

Excepciones

Otros

